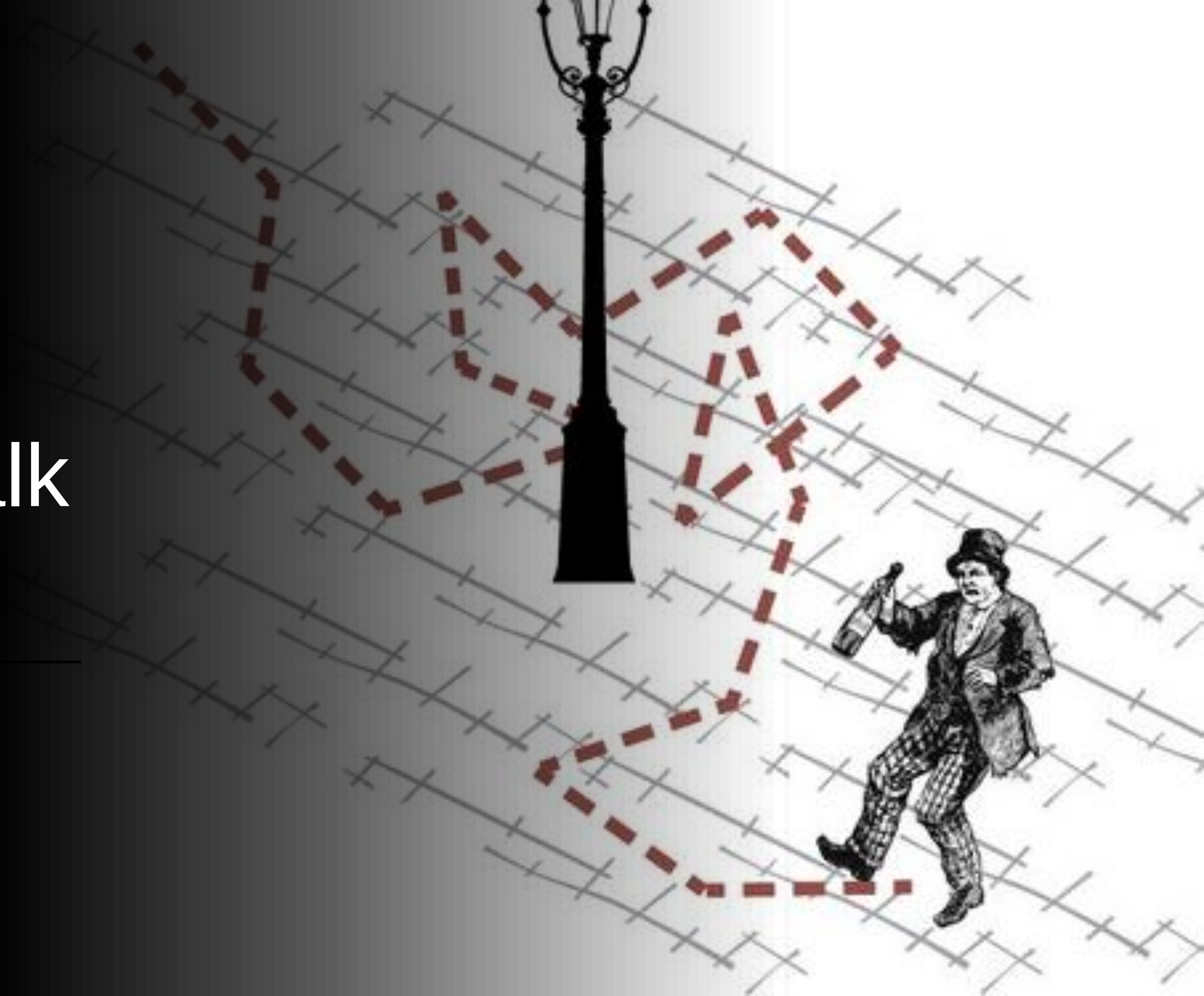
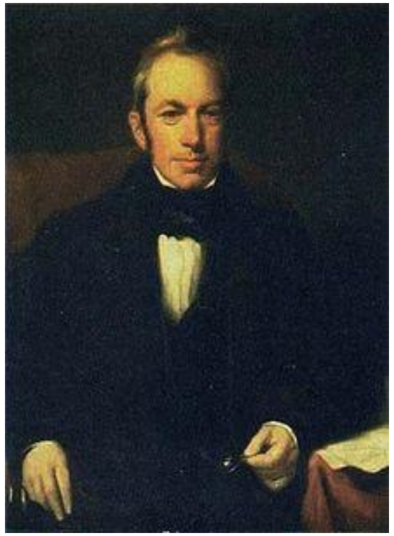
# Diffusion & Random Walk model

Minho Lee

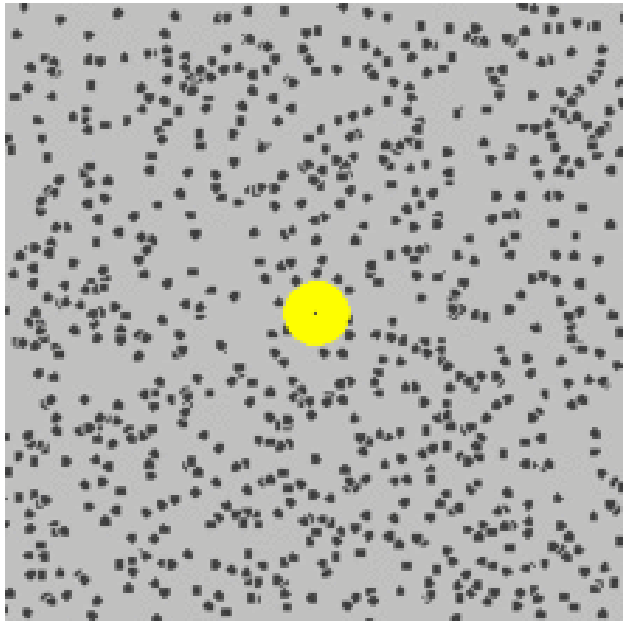


# What is Diffusion?

# Brownian motion (1827)

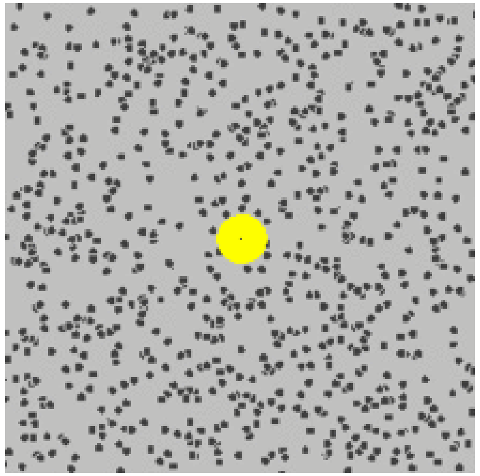


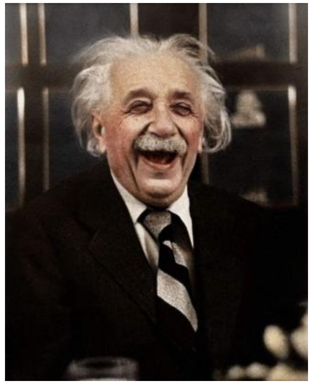
**Robert Brown**
**(식물학자)**



- **불규칙성**: 입자의 운동이 불규칙하고 예측할 수 없습니다.
- **연속성과 비미분성**: 꽃가루 입자의 경로는 연속적이지만, 어느 지점에서도 접선을 가지지 않는다 (즉, 미분 불가능하다).
- **독립성**: 입자들이 서로 가까이 있어도 독립적으로 움직인다. 꽃가루 입자 간의 상호작용은 순수하게 탄성적인 것으로 보인다.
- **액체 점성의 영향**: 액체의 점성이 감소하면 꽃가루 입자의 운동이 더 활발해진다.
- **입자 크기의 영향**: 입자의 크기가 감소하면 운동이 더 활발해집니다.
- **온도의 영향**: 주변 온도가 증가하면 운동이 더 활발해집니다.
- **지속성**: 꽃가루 입자의 운동이 멈추지 않습니다.

# Theoretical approach to describe Brownian motion



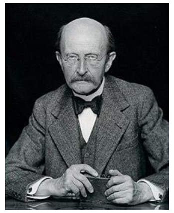


**Albert Einstein (1905)** [Link]

- 꽃가루 입자는 끊임없이 움직이는 용매 분자와 충돌하여 움직인다.
- 이를 **확률 모델** ($P(x,t)$) 로 기술 가능하다.
- $\partial_t P(x,t) = D\ \partial_x^2 P(x,t)$

**Paul Langevin (1908)** [Link]

- 꽃가루는 두 가지 힘의 영향을 받는다;
  1. 점성에 의한 **dragging force** ($-\lambda \boldsymbol{v}$)
  2. 용매 분자에 의한 **random force** ($\boldsymbol{\eta}$)
- $m\dot{\boldsymbol{v}} = -\lambda \boldsymbol{v} + \boldsymbol{\eta}$

[Link] [Link]

**Adriaan Fokker (1914), Max Planck (1917) and Marian Smoluchowski (1916)** [Link]

- Langevin equation에 **외력** ($-\boldsymbol{\nabla} U$) 까지 고려한 경우에 대해, Einstein equation과 유사한 결과 식을 도출할 수 있다.

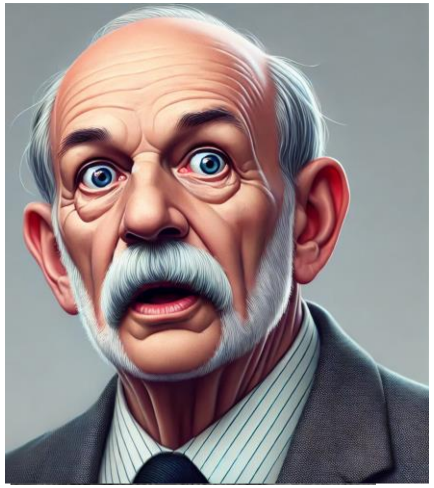

식 유도과정은 생략합니다.

- $m\dot{\boldsymbol{v}} = -\lambda \boldsymbol{v} + \boldsymbol{\eta} - \boldsymbol{\nabla} U$
- $\partial_t P(x,t) = [D(\partial_x^2 + \partial_x(\beta U)\partial_x)]P(x,t)$

Nonequilibrium Statistical Mechanics: Zwanzig, Robert

# Random walk model

**Karl Pearson (1905)**



[Link]

The Problem of the Random Walk.

CAN any of your readers refer me to a work wherein I should find a solution of the following problem, or failing the knowledge of any existing solution provide me with an original one? I should be extremely grateful for aid in the matter.

A man starts from a point O and walks $l$ yards in a straight line; he then turns through any angle whatever and walks another $l$ yards in a second straight line. He repeats this process $n$ times. I require the probability that after these $n$ stretches he is at a distance between $r$ and $r+\delta r$ from his starting point, O.

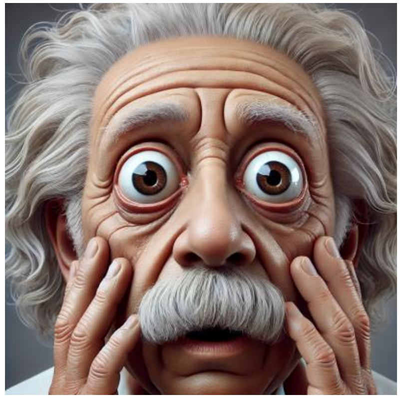
The problem is one of considerable interest, but I have only succeeded in obtaining an integrated solution for *two* stretches. I think, however, that a solution ought to be found, if only in the form of a series in powers of $1/n$, when $n$ is large.

KARL PEARSON.

The Gables, East Ilsley, Berks.

**Francis Galton**

**Galton board (1889)**

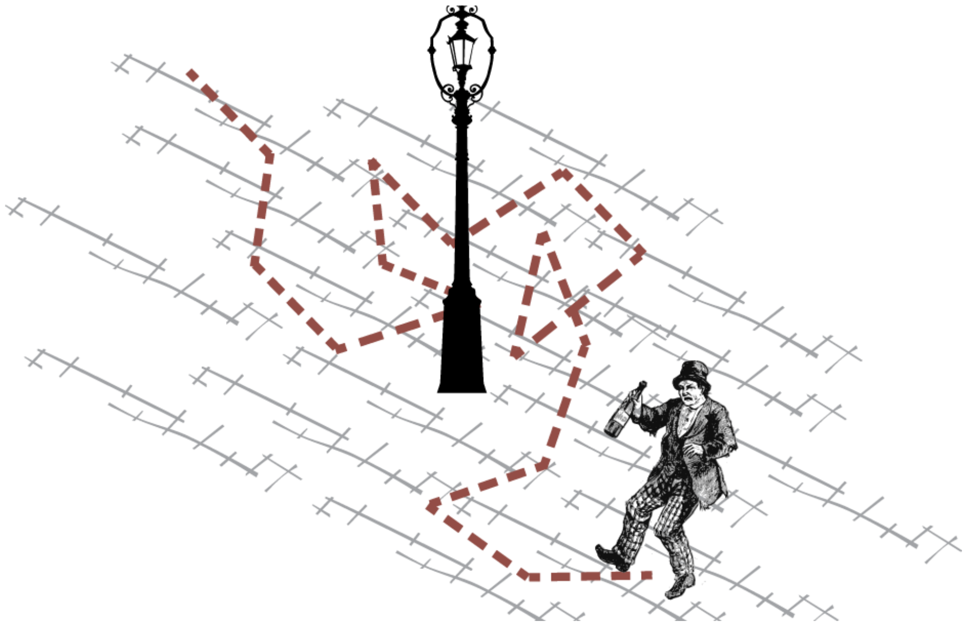


**Albert Einstein (1905)** [Link]

- 꽃가루 입자는 끊임없이 움직이는 용매 분자와 충돌하여 움직인다.
- 이를 확률 모델로 기술 가능하다.
- $\partial_t P(x,t) = D\, \partial_x^2 P(x,t)$

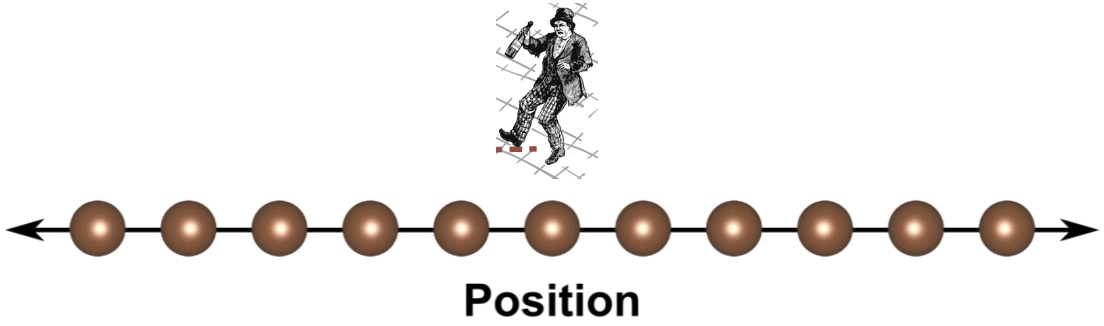
$$\rightarrow P(x,t) = \frac{1}{\sqrt{4\pi D t}} e^{-\frac{x^2}{4Dt}}$$

# Today’s goals for the lesson

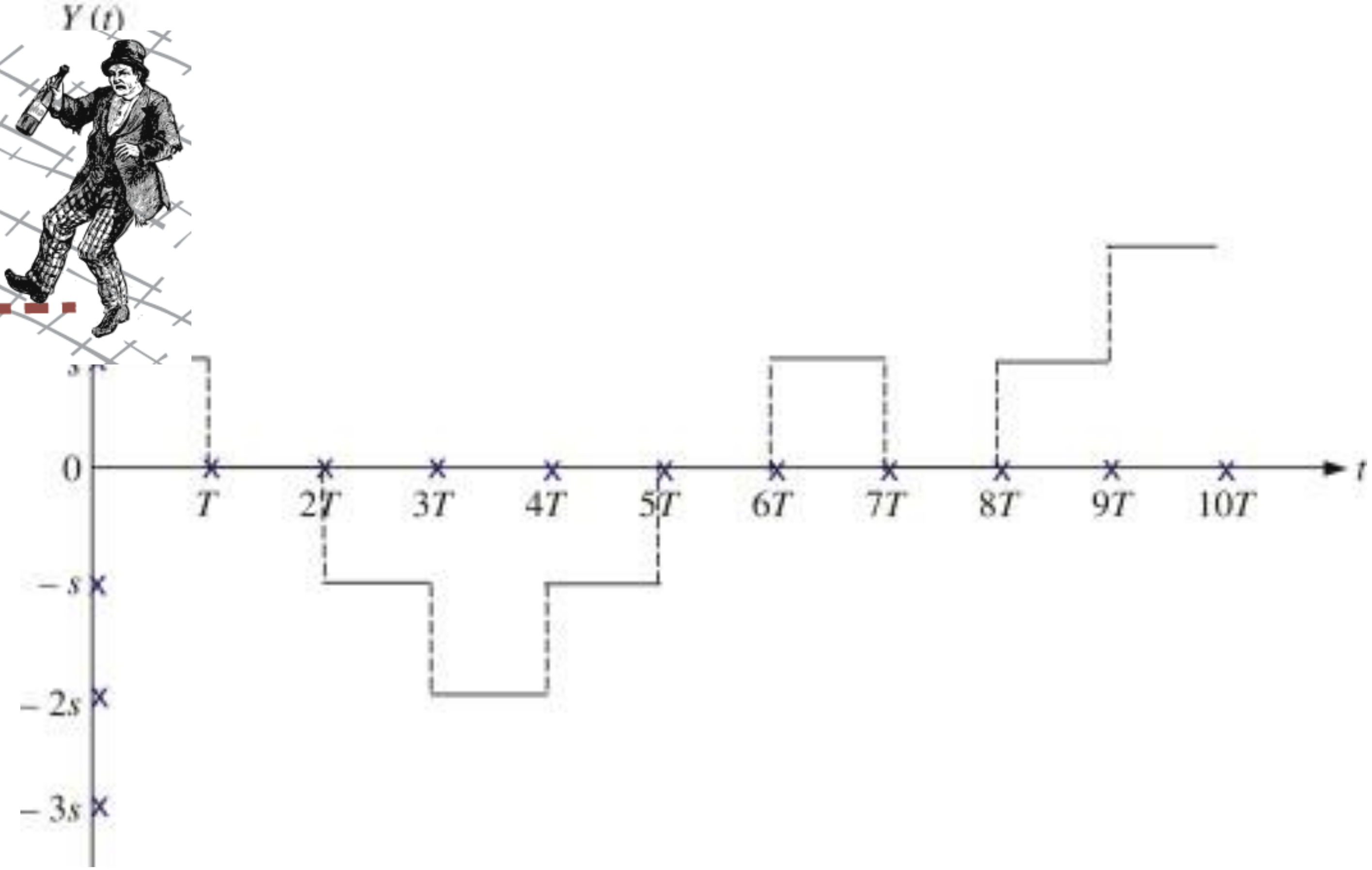


Derive the **Smoluchowski equation** from the **Random walker model**.

# Random walk model



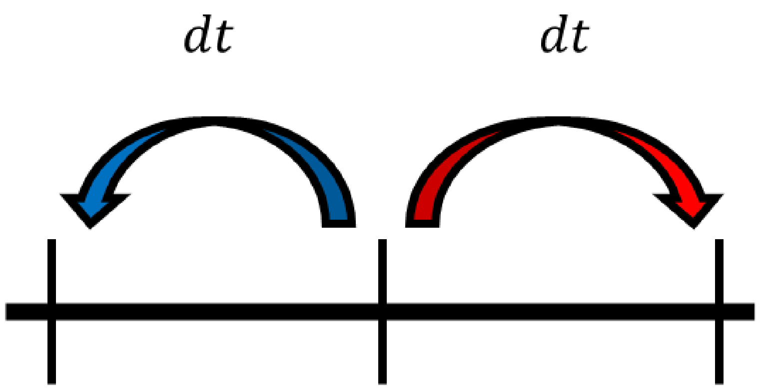

1. 1차원 격자 공간 (1D lattice space)
2. 술에 취한 사람(Random walker)은 한번에 한 칸 씩만 이동 가능한 상황



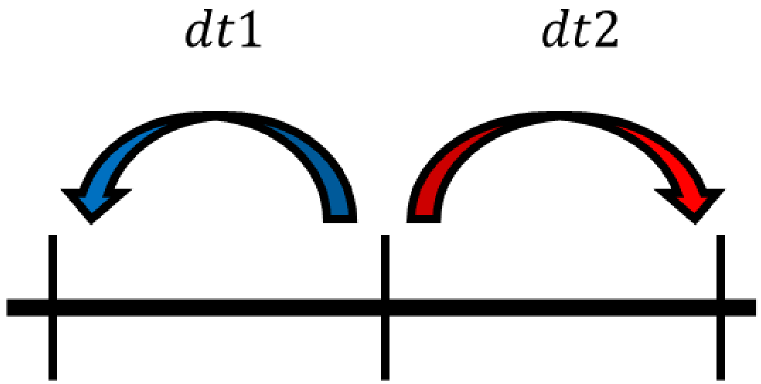

**Questions:**

1. 움직임 사이의 시간 간격은 어떻게 되나요?
2. 왼쪽으로 움직일지, 오른쪽으로 움직일지는 어떻게 결정하나요?

# Continuous Time Random Walk (CTRW) model

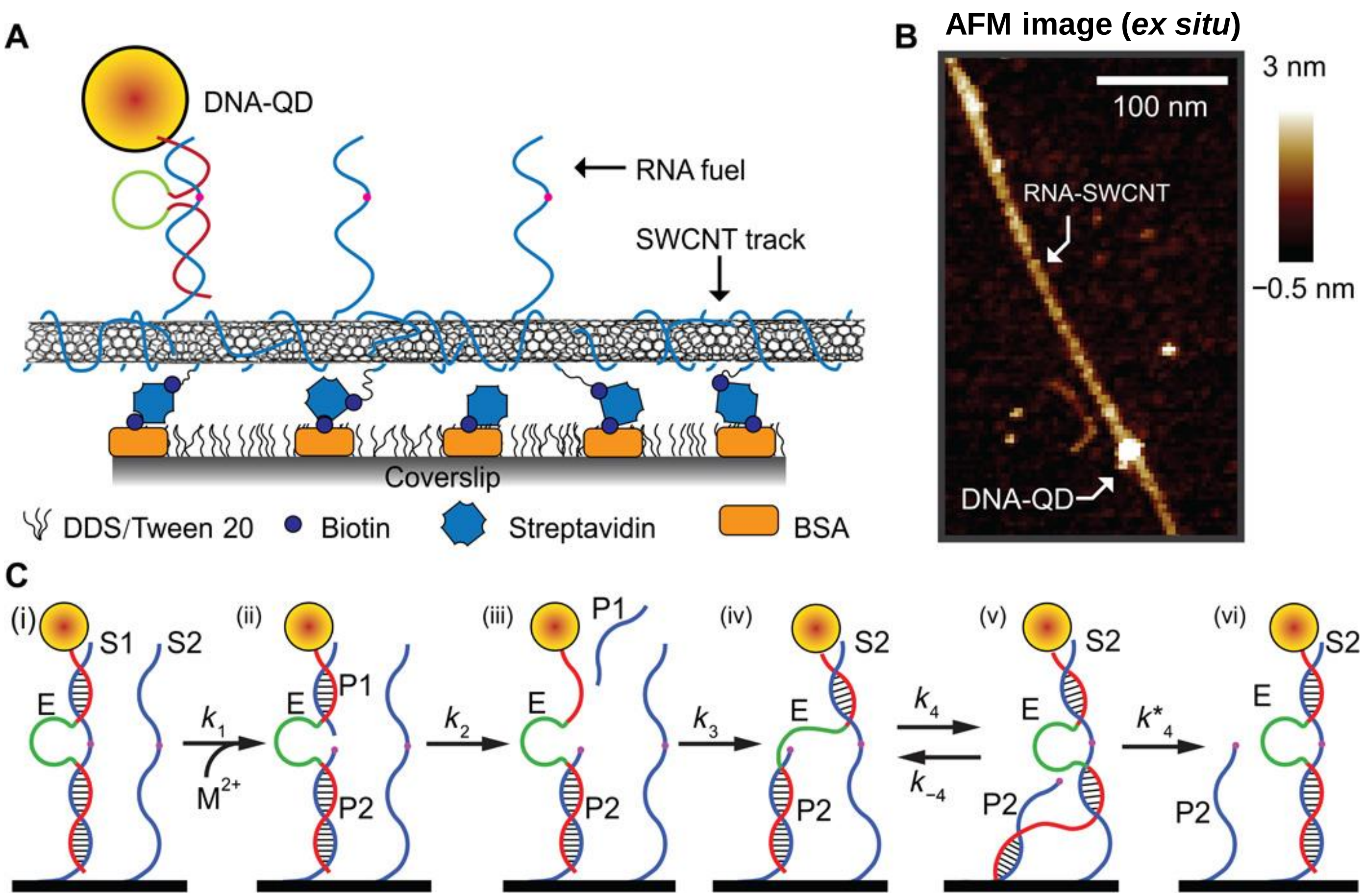


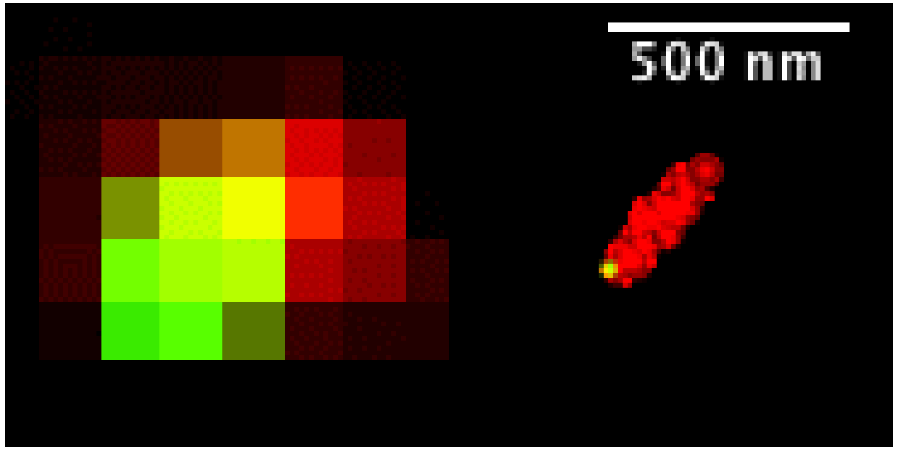

**[Classical random Walk]**
Coin toss & fixed dt



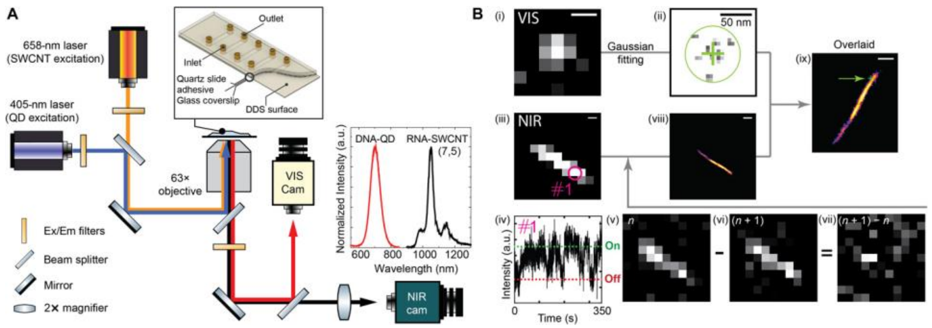

**[Continuous Time Random Walk]**
Time competition & mutable dt

# Continuous Time Random Walk (CTRW) model




**VIS/NIR subdiffraction imaging (*in situ*)**



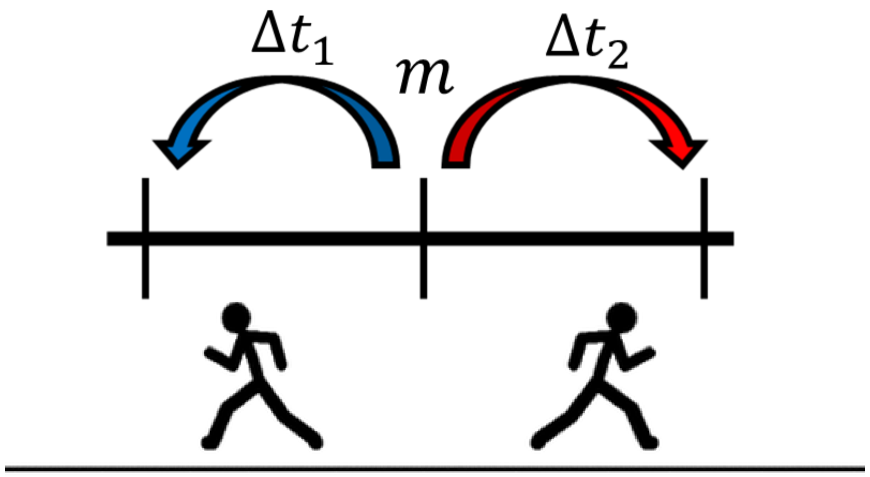

Pan et al., Science Advances, 3, 1, e1601600 (2017)

CTRW = **realistic model** that can handle a **wider variety of cases** compared to classical random walk.

# Basic concept of CTRW model

**Capital letter: competition**

**Small letter: w/o competition**




$\psi_m^L(t)$ $\psi_m^R(t)$

$\psi_m^{L(R)}(t)$ :

Waiting time distribution that the jump from site m to left(right) side occurs in absence of jump to right(left) side.

$$\Psi_m^{L(R)}(t) = \psi_m^{L(R)}(t) \int_t^\infty d\tau\, \psi_m^{R(L)}(\tau) :$$

Waiting time distribution that the jump from site m to left(right) side occurs.

$\Psi_m(t) = \Psi_m^L(t) + \Psi_m^R(t)$ : Normalized waiting time distribution that the jump from site m occurs.

$\Psi_m(t) = \Psi(t)$ : Assume that waiting time distribution is the same for all sites except for the boundaries.

# Basic concept of CTRW model

$f_n(t|m)\mathrm{dt}$ :

Probability that the random walker starts at a site m at time 0 and arrives at a site n between time t ~ t + dt

$$f_n(t|m) = \left(\int_0^t d\tau\, \Psi^R(\tau) f_{n-1}(t-\tau|m) + \int_0^t d\tau\, \Psi^L(\tau) f_{n+1}(t-\tau|m)\right) + \delta_{nm}\delta(t)$$

$$\hat{f}_n(u|m) \equiv \int_0^\infty dt\, e^{-ut}\, f_n(t|m) = \left(\widehat{\Psi}^R(u)\hat{f}_{n-1}(u|m) + \widehat{\Psi}^L(u)\hat{f}_{n+1}(u|m)\right) + \delta_{nm}$$

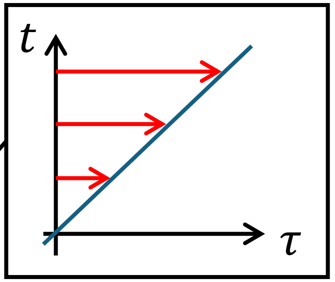
$$= \widehat{\Psi}(u)\left(R\hat{f}_{n-1}(u|m) + L\hat{f}_{n+1}(u|m)\right) + \delta_{nm}$$

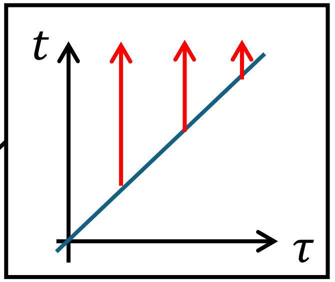
$\psi^L(t)$와 $\psi^R(t)$가
같은 distribution을 따른다면.

$$\widehat{\Psi}^L(0) = L \ \ \& \ \ \widehat{\Psi}^R(0) = R$$

---

$$\int_0^\infty dt\, e^{-ut}\left[\int_0^t d\tau\, a(t-\tau)b(\tau)\right] = \hat{a}(u)\hat{b}(u)$$




$$\int_0^\infty dt\, e^{-ut}\left[\int_0^t d\tau\, a(t-\tau)b(\tau)\right] = \int_0^\infty dt \int_0^t d\tau\, \left[e^{-ut}a(t-\tau)b(\tau)\right] = \int_0^\infty d\tau \int_\tau^\infty dt\, \left[e^{-ut}a(t-\tau)b(\tau)\right]$$

$$= \int_0^\infty d\tau\, b(\tau)\int_\tau^\infty dt\, \left[e^{-ut}a(t-\tau)\right] = \int_0^\infty d\tau\, b(\tau)\int_0^\infty dt\, \left[e^{-u(t+\tau)}a(t)\right] = \int_0^\infty d\tau\, e^{-u\tau}b(\tau) \times \hat{a}(u) = \hat{b}(u)\hat{a}(u)$$

# Basic concept of CTRW model

$f_n(t|m)\mathrm{dt}$ :

Probability that the random walker starts at a site m at time 0 and arrives at a site n between time t ~ t + dt

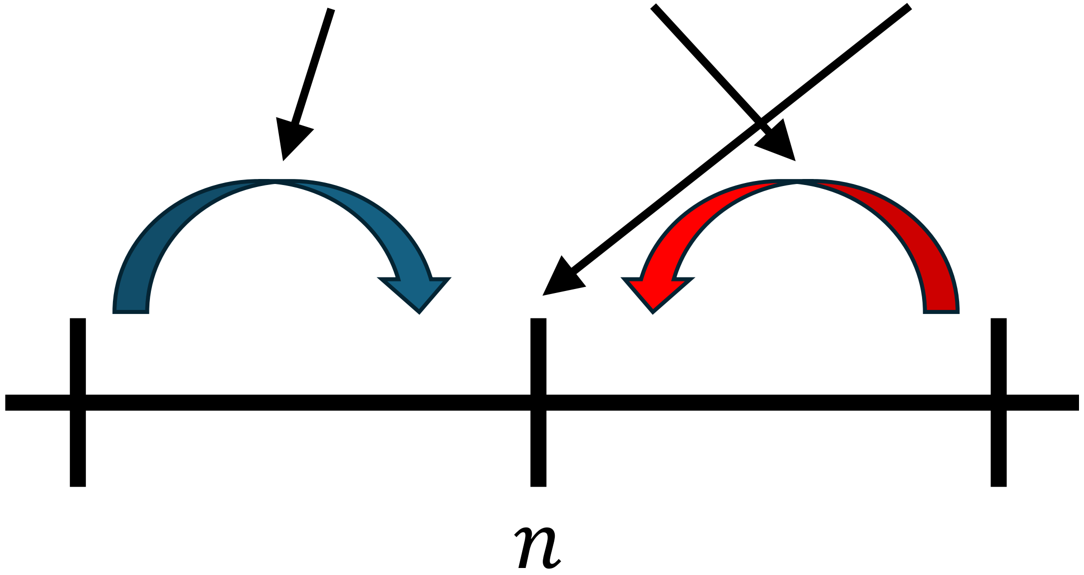
$$f_n(t|m) = \left(\int_0^t d\tau\,\Psi^R(\tau) f_{n-1}(t-\tau|m) + \int_0^t d\tau\,\Psi^L(\tau) f_{n+1}(t-\tau|m)\right) + \delta_{nm}\delta(t)$$

$$\hat{f}_n(u|m) \equiv \int_0^\infty dt\, e^{-ut}\, f_n(t|m) = \left(\widehat{\Psi}^R(u)\hat{f}_{n-1}(u|m) + \widehat{\Psi}^L(u)\hat{f}_{n+1}(u|m)\right) + \delta_{nm}$$

$$= \widehat{\Psi}(u)\left(R\hat{f}_{n-1}(u|m) + L\hat{f}_{n+1}(u|m)\right) + \delta_{nm}$$

$\psi^L(t)$와 $\psi^R(t)$가
같은 distribution을 따른다면.

$$\widehat{\Psi}^L(0) = L \;\;\&\;\; \widehat{\Psi}^R(0) = R$$




1. Lattice site $n-1$에 시간 $\tau$에 도착한 random walker가 $t-\tau$ 의 시간 이후 오른쪽으로 움직여 lattice site $n$ 에 도달하는 경우
2. Lattice site $n+1$에 시간 $\tau$에 도착한 random walker가 $t-\tau$ 의 시간 이후 왼쪽으로 움직여 lattice site $n$ 에 도달하는 경우
3. 시간 $0$에 lattice site $n$에 random walker가 생성된 경우. 이 경우에는 공간에 생성되는 순간이 도착하는 순간이 되어버린다.

# Basic concept of CTRW model

$p_n(t|m)$ :

Probability that the random walker starts at a site m at time 0 and is found at a site n at time t.

$$\hat{p}_n(u|m) = \frac{1-\hat{\Psi}(u)}{u}\hat{f}_n(u|m)$$

$$\boxed{p_n(t|m) = \int_0^t d\tau\, S(\tau) f_n(t-\tau|m) \qquad S(t) = \int_0^t d\tau\ \Psi(\tau) \rightarrow \hat{S}(u) = \frac{1-\hat{\Psi}(u)}{u}}$$

$$\hat{f}_n(u|m) = \hat{\Psi}(u)\left(R\hat{f}_{n-1}(u|m) + L\hat{f}_{n+1}(u|m)\right) + \delta_{nm}$$

$$\rightarrow \hat{\dot{p}}_n(u|m) = \hat{\mathrm{D}}(u)\left(R\hat{p}_{n-1}(u|m) + L\hat{p}_{n+1}(u|m) - \hat{p}_n(u|m)\right)$$

: **G**eneralized **M**aster **E**quation (GME) of random walker.

$$\hat{\mathrm{D}}(u) = \frac{u\hat{\Psi}(u)}{1-\hat{\Psi}(u)}$$ : discrete space Diffusion Kernel

$$\hat{\dot{p}}(n,u|m) = u\hat{p}(n,u|m) - \delta_{n,m}$$

$$\left(\because \int_0^\infty dt\, e^{-ut}\,\dot{a}(t) = \left(a(t)e^{-ut}\right)\Big|_{t=0}^{\infty} + u\int_0^\infty dt\, e^{-ut}\, a(t) = -a(0) + u\hat{a}(u)\right)$$

# Basic concept of CTRW model

$$\hat{p}_n(u|m) = \frac{1-\widehat{\Psi}(u)}{u}\hat{f}_n(u|m) \quad \rightarrow u\hat{p}_n(u|m) - \delta_{n,m} = \left(1-\widehat{\Psi}(u)\right)\hat{f}_n(u|m) - \delta_{n,m} = \dot{\hat{p}}_n(u|m)$$

$$\left(1-\widehat{\Psi}(u)\right)\hat{f}_n(u|m) - \delta_{n,m} = \left(\hat{f}_n(u|m) - \delta_{n,m}\right) - \widehat{\Psi}(u)\hat{f}_n(u|m)$$

$$\hat{f}_n(u|m) = \widehat{\Psi}(u)\left(R\hat{f}_{n-1}(u|m) + L\hat{f}_{n+1}(u|m)\right) + \delta_{nm}$$

$$= \widehat{\Psi}(u)\left(R\hat{f}_{n-1}(u|m) + L\hat{f}_{n+1}(u|m) - \hat{f}_n(u|m)\right)$$

$$\hat{p}_n(u|m) = \frac{1-\widehat{\Psi}(u)}{u}\hat{f}_n(u|m)$$

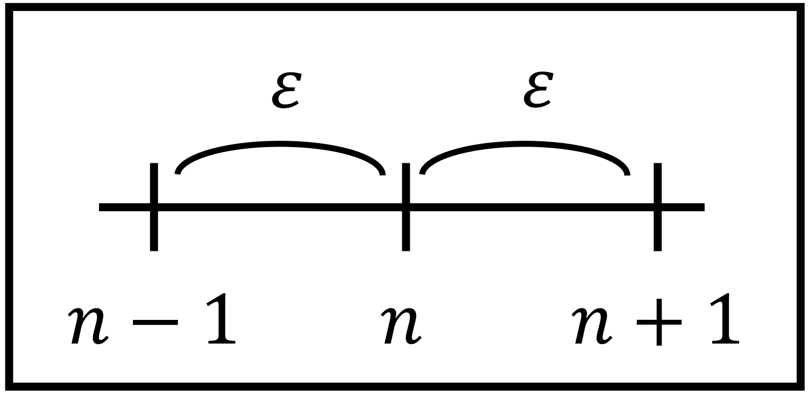
$$= \frac{u\widehat{\Psi}(u)}{1-\widehat{\Psi}(u)}\left(R\hat{p}_{n-1}(u|m) + L\hat{p}_{n+1}(u|m) - \hat{p}_n(u|m)\right)$$

$$\widehat{\mathrm{D}}(u) = \frac{u\widehat{\Psi}(u)}{1-\widehat{\Psi}(u)}$$

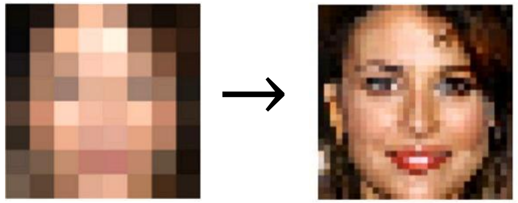
$$= \widehat{D}(u)\left(R\hat{p}_{n-1}(u|m) + L\hat{p}_{n+1}(u|m) - \hat{p}_n(u|m)\right)$$

# Discrete to Continuous

$$\dot{\hat{p}}_n(u|m) = \hat{\mathrm{D}}(u)\big(R\hat{p}_{n-1}(u|m) + L\hat{p}_{n+1}(u|m) - \hat{p}_n(u|m)\big)$$




$$\lim \varepsilon \to 0$$



$$\dot{\hat{P}}(x,u|x_0) = \hat{\mathcal{D}}(u)(\,\partial_x^2 + \beta U'(x)\partial_x)\hat{P}(x,u|x_0)$$

$\hat{\mathcal{D}}(u) = \frac{\varepsilon^2}{2}\hat{D}(u)$ $\qquad$ $\beta U'(x) = 2(L-R)/\varepsilon$

$\hat{p}_n(u|m) = \varepsilon \hat{P}(x,u|x_0)$ $\qquad$ $x = \varepsilon n,$ $\qquad$ $x_0 = \varepsilon m$

**Diffusion Kernel** value needs to be **invariant** to lattice constant $\varepsilon$.

$\boldsymbol{\hat{\mathcal{D}}(u) = \frac{\varepsilon^2}{2}\hat{D}(u)}$**: diffusion kernel**

# Discrete to Continuous

$$\dot{\hat{p}}_n(u|m) = \widehat{\mathrm{D}}(u)\big(R\hat{p}_{n-1}(u|m) + L\hat{p}_{n+1}(u|m) - \hat{p}_n(u|m)\big)$$

$$\hat{p}_n(u\,|\,m) = \varepsilon \hat{P}(x,u\,|\,x_0) \qquad x = \varepsilon n, \quad x_0 = \varepsilon m$$

$$\hat{P}(x,u|x_0)$$

$$= (R+L)\hat{P}(x,u|x_0)$$

$$\dot{\hat{P}}(x,u|x_0) = \widehat{\mathrm{D}}(u)\left(R\hat{P}(x-\varepsilon,u|x_0) + L\hat{P}(x+\varepsilon,u|x_0) - \hat{P}(x,u|x_0)\right)$$

$$= \widehat{\mathrm{D}}(u)\left(R\hat{P}(x-\varepsilon,u|x_0) + L\hat{P}(x+\varepsilon,u|x_0) - R\hat{P}(x,u|x_0) - L\hat{P}(x,u|x_0)\right)$$

Taylor expansion

$$\boxed{\begin{aligned}\hat{P}(x+\varepsilon,u|x_0) &\cong \hat{P}(x,u|x_0) + \varepsilon\,\partial_x\hat{P}(x,u|x_0) + \varepsilon^2/2\ \partial_x^2\hat{P}(x,u|x_0) + \cdots \\ \hat{P}(x-\varepsilon,u|x_0) &\cong \hat{P}(x,u|x_0) - \varepsilon\,\partial_x\hat{P}(x,u|x_0) + \varepsilon^2/2\ \partial_x^2\hat{P}(x,u|x_0) + \cdots\end{aligned}}$$

$$\dot{\hat{P}}(x,u|x_0) = \widehat{\mathrm{D}}(u)\left(\varepsilon\,(L-R)\partial_x\hat{P}(x,u|x_0) + (L+R)\,\varepsilon^2/2\ \partial_x^2\hat{P}(x,u|x_0)\right)$$

$$= \varepsilon^2\widehat{\mathrm{D}}(u)/2\left(2(L-R)/\varepsilon \times \partial_x\hat{P}(x,u|x_0) + (L+R)\,\partial_x^2\hat{P}(x,u|x_0)\right)$$

# Discrete to Continuous

$$u\hat{\dot{P}}(x,u|x_0) = u\widehat{\mathrm{D}}(u)\left(R\hat{P}(x-\varepsilon,u|x_0) + L\hat{P}(x+\varepsilon,u|x_0) - R\hat{P}(x,u|x_0) - L\hat{P}(x,u|x_0)\right)$$

$\lim u \to 0$

**Tauberian theorem:** $\lim_{u\to 0} u\hat{f}(u) = \lim_{t\to\infty} f(t)$

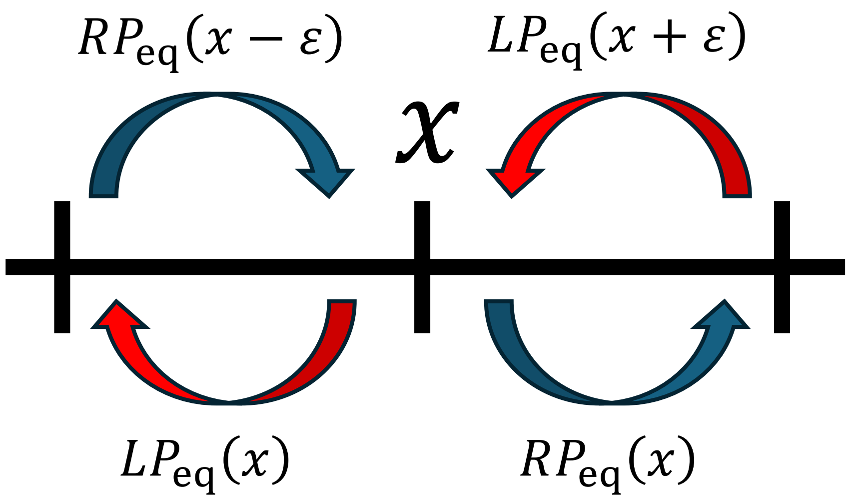
**(or Final Value Theorem)**

$$u\hat{f}(u) = u\int_0^\infty dt\, e^{-ut} f(t) = -e^{-ut} f(t)\Big|_0^\infty + \int_0^\infty dt\, e^{-ut} f'(t)$$

$$\therefore \lim_{u\to 0} u\hat{f}(u) = f(0) + \int_0^\infty dt\, f'(t) = f(\infty) = \lim_{t\to\infty} f(t)$$

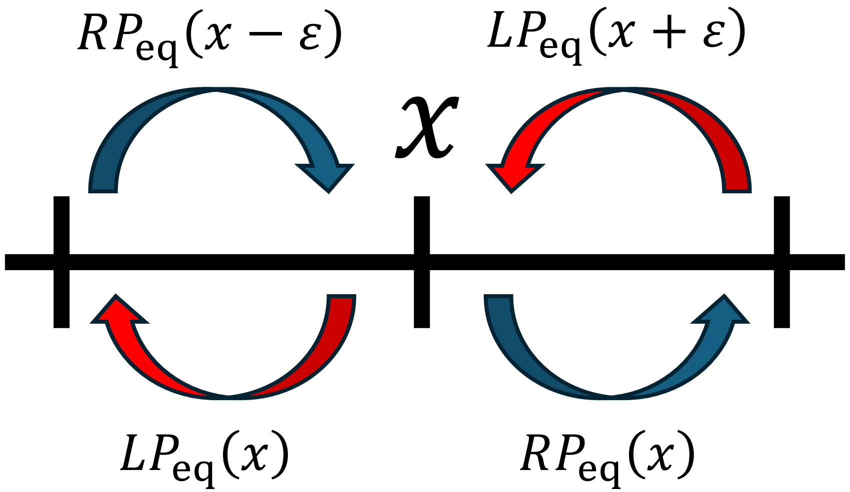
**정확한 증명은 아니니, 맹신하면 안됩니다.**
**궁금하면 아래의 내용을 보시길 바랍니다.**
**[link 1], [link 2], [link 3]**

$$\lim_{t\to\infty} \dot{P}(x,t|x_0) = 0 = u\widehat{\mathrm{D}}(0)\left(RP_{\mathrm{eq}}(x-\varepsilon) + LP_{\mathrm{eq}}(x+\varepsilon) - RP_{\mathrm{eq}}(x) - LP_{\mathrm{eq}}(x)\right)$$




들어온 만큼, 나가는 상황이 되기에, equilibrium에 도달한다.

# Discrete to Continuous




$$\frac{L-R}{\varepsilon}=\frac{LP_{\mathrm{eq}}(x)-RP_{\mathrm{eq}}(x)}{\varepsilon P_{\mathrm{eq}}(x)}=\frac{LP_{\mathrm{eq}}(x)-LP_{\mathrm{eq}}(x+\varepsilon)}{\varepsilon P_{\mathrm{eq}}(x)}=-\frac{P_{\mathrm{eq}}(x+\varepsilon)-P_{\mathrm{eq}}(x)}{\varepsilon}\times\frac{L}{P_{\mathrm{eq}}(x)}=\frac{LP'_{\mathrm{eq}}(x)}{P_{\mathrm{eq}}(x)}$$

$$+\;\frac{L-R}{\varepsilon}=\frac{LP_{\mathrm{eq}}(x)-RP_{\mathrm{eq}}(x)}{\varepsilon P_{\mathrm{eq}}(x)}=\frac{RP_{\mathrm{eq}}(x-\varepsilon)-RP_{\mathrm{eq}}(x)}{\varepsilon P_{\mathrm{eq}}(x)}=-\frac{P_{\mathrm{eq}}(x)-P_{\mathrm{eq}}(x-\varepsilon)}{\varepsilon}\times\frac{R}{P_{\mathrm{eq}}(x)}=\frac{RP'_{\mathrm{eq}}(x)}{P_{\mathrm{eq}}(x)}$$

$$2\frac{L-R}{\varepsilon}=-\frac{P'_{\mathrm{eq}}(x)}{P_{\mathrm{eq}}(x)}\rightarrow P_{\mathrm{eq}}(x)\sim\exp\left(-\frac{2(L-R)}{\varepsilon}x\right)$$

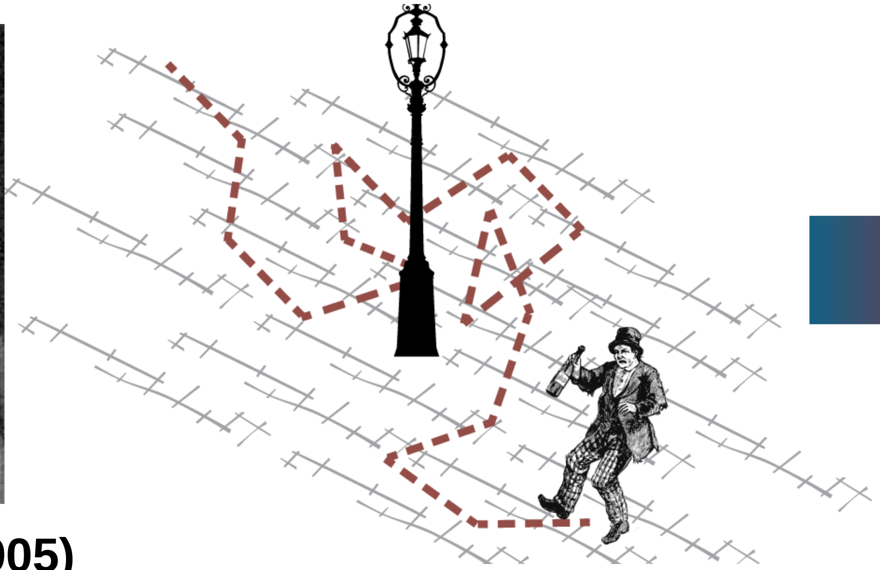
$$2(L-R)/\varepsilon=\beta U'(x)\qquad\left(\because P_{\mathrm{eq}}(x)\sim e^{-\beta U(x)}\right)$$

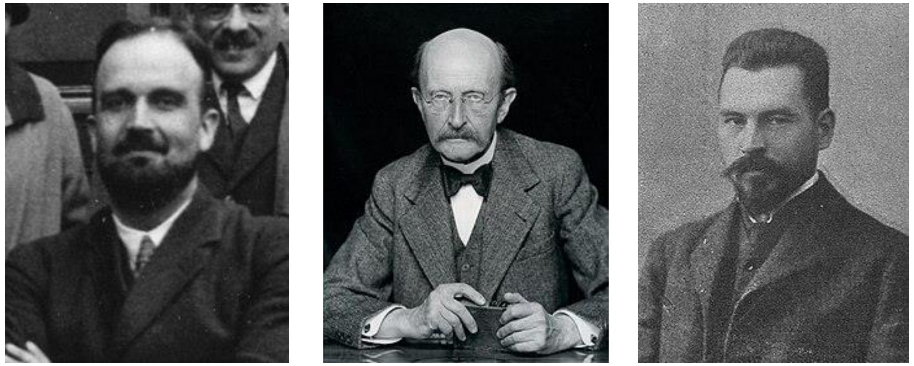
# Discrete to Continuous

$$\hat{\dot{P}}(x,u|x_0) = \varepsilon^2\hat{\mathrm{D}}(u)/2\left(2(L-R)/\varepsilon \times \partial_x \hat{P}(x,u|x_0) + \partial_x^2 \hat{P}(x,u|x_0)\right)$$

$$= \underbrace{\hat{\mathcal{D}}(u)(\beta U'(x)\partial_x + \partial_x^2)}_{\textbf{Fokker-Planck Operator}}\hat{P}(x,u|x_0)$$

**Karl Pearson (1905)**





**Adriaan Fokker (1914), Max Planck (1917) and Marian Smoluchowski (1916)**

- $\partial_t P(x,t) = [D(\partial_x^2 + \partial_x(\beta U)\partial_x)]P(x,t)$

# Application: Universality!

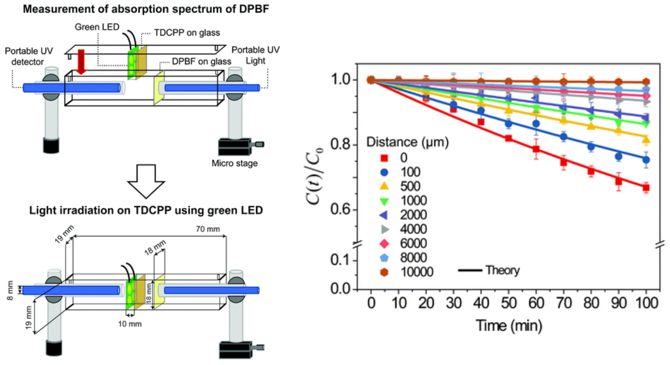
$$\dot{\hat{P}}(x,u|x_0) = \varepsilon^2 \hat{\mathrm{D}}(u)/2 \left(2(L-R)/\varepsilon \times \partial_x \hat{P}(x,u|x_0) + \partial_x^2 \hat{P}(x,u|x_0)\right)$$

$$= \underbrace{\hat{\mathcal{D}}(u)(\beta U'(x)\partial_x + \partial_x^2)}_{\textbf{Fokker-Planck Operator}} \hat{P}(x,u|x_0)$$

Kinesin Motor Protein 3D Animation

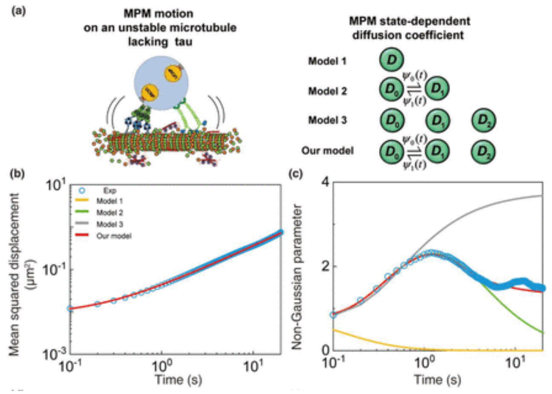
The MAPK Signaling Pathway - YouTube

# Application: Universality!



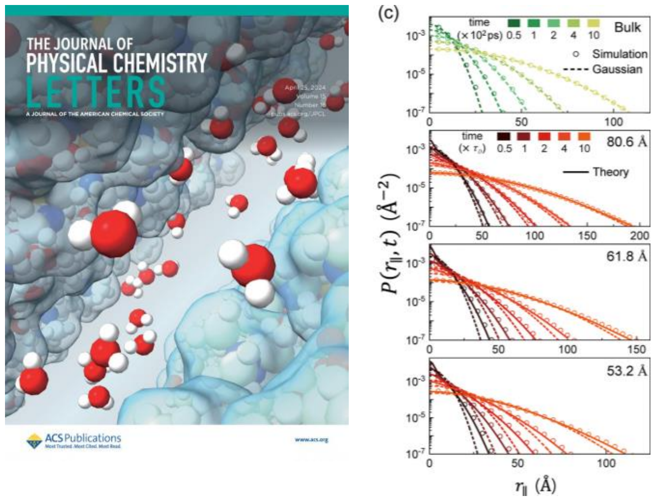

강력한 oxidant로 사용 가능한 singlet oxygen의 diffusion에 대한 연구

Wang *et. al.*, *PCCP*, 22, 21664 (2020)



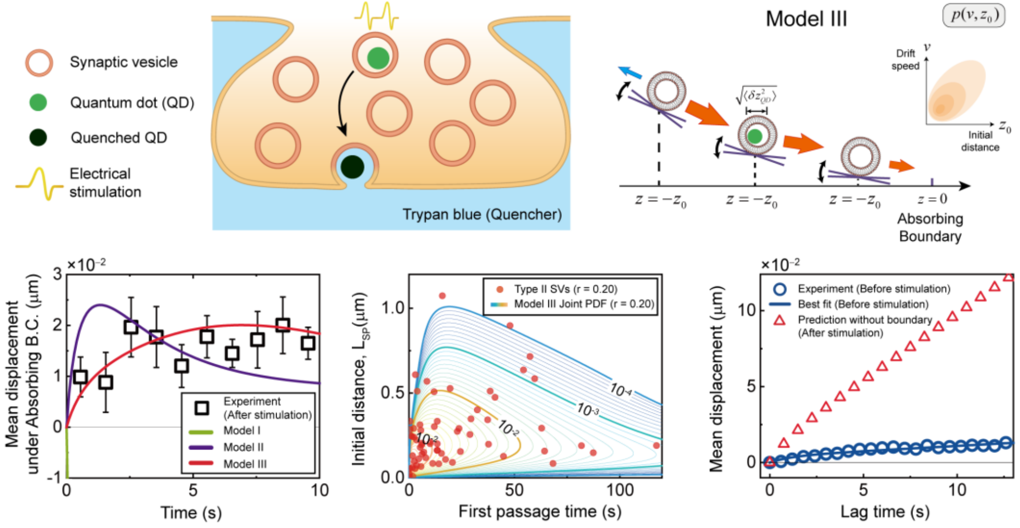

Neuronal communication에 지대한 영향을 미치는 synaptic vesicle의 transport dynamics에 대한 연구

Lee *et. al.*, Chem. Biomed. Imaging, 2, 5, 362-373 (2024)




세포막 사이에 갇힌 물 분자의 diffusion에 대한 연구

Lee *et. al.*, *JPCL*, 15, 16, 4437-4443 (2024)




Synaptic vesicle의 transport dynamics에 대한 연구

# Motivation: why do we need to know that?

Ptolemy
(100-170)

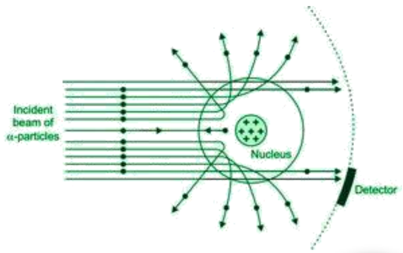
Tycho Brahe
(1546-1601)

Mathematics is the language in which God has written the universe.

Galileo Galilei
(1564-1642)

Johannes Kepler
(1571-1630)

Isaac Newton
(1643-1727)





Elon Musk
(1971-    )

Ernest Rutherford
(1871-1937)

Planet Neptune
(predicted by Newtonian dynamics)

**1단계**

**관측하는 단계**

**2단계**

**유의미한 법칙을 찾는 단계**

**(이유는 모름)**

**3단계**

**수식으로 설명하는 단계**

**(Physical meaning)**

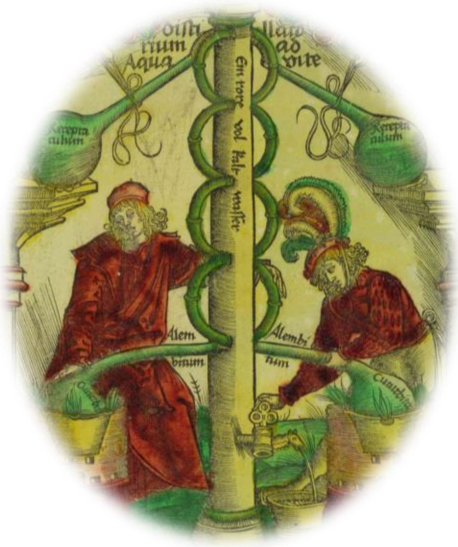
**4단계**

**3단계에서 얻은 이론을 활용하는 단계**

**(작은 것부터 큰 것까지 모두 맞출 수 있다.)**

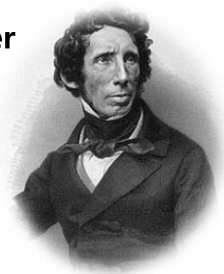
# Motivation: why do we need to know that?

Chemical Reaction

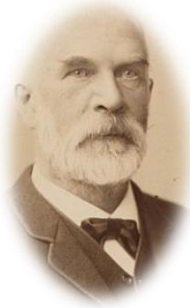



Alchemists
(300 AD – Medieval Europe)

Antoine Lavoisier
(1743-1794)



Friedrich Wöhler
(1800-1882)



Cato M. Guldberg
(1836-1902)

Peter Waage
(1833-1900)

van’t Hoff Jr.
(1852-1911)

Fritz Haber
(1868-1934)

$$N_2 + 3H_2 \rightleftharpoons 2NH_3$$

**1단계**

**관측하는 단계**

**2단계**

**유의미한 법칙을 찾는 단계**

**(이유는 모름)**

**3단계**

**수식으로 설명하는 단계**

**(Physical meaning)**

**4단계**

**3단계에서 얻은 이론을 활용하는 단계**

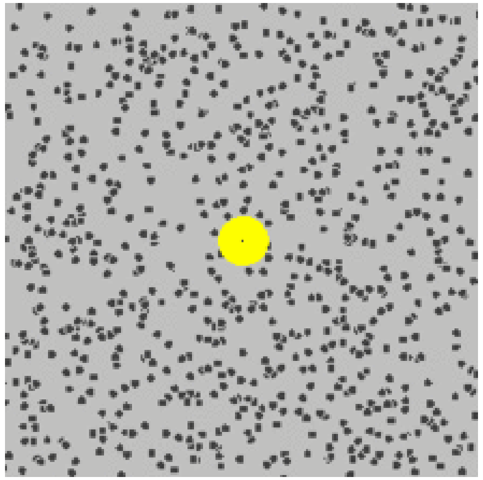
**(작은 것부터 큰 것까지 모두 맞출 수 있다.)**

# Motivation: why do we need to know that?

Robert Brown
(1773-1858)

**1단계**

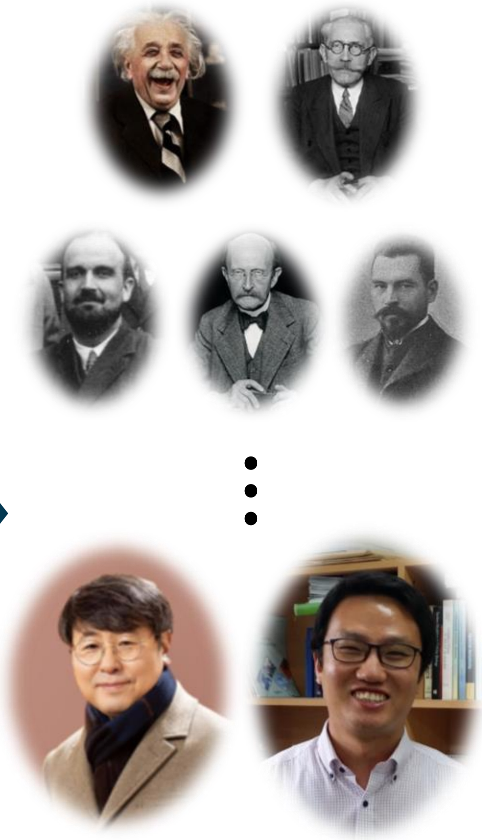
**관측하는 단계**



**2단계**

**유의미한 법칙을 찾는 단계**

**(이유는 모름)**



**3단계**

**수식으로 설명하는 단계**

**(Physical meaning)**

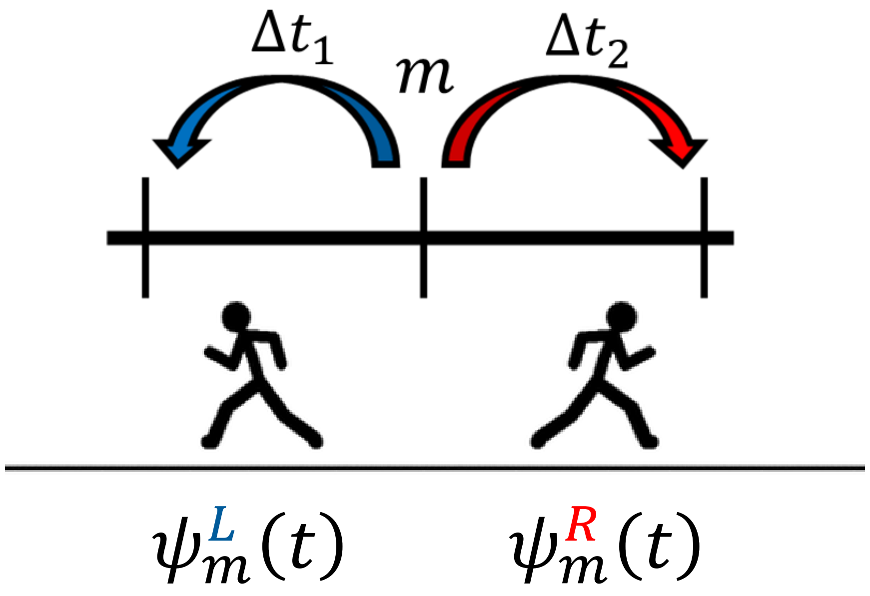
**4단계**

**3단계에서 얻은 이론을 활용하는 단계**

**(작은 것부터 큰 것까지 모두 맞출 수 있다.)**


*Thank you for your attention*

# Supplementary Material: How to make simulation code?



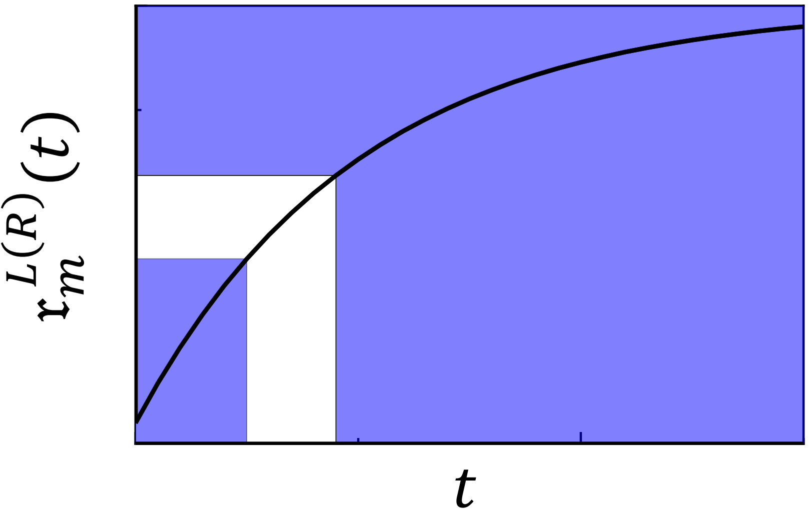

$\psi_m^{L(R)}(t)$ :

Waiting time distribution that the jump from site m to left(right) side occurs in absence of jump to right(left) side.

$$\mathfrak{r}_m^{L(R)}(t) \equiv \int_0^t d\tau\, \psi_m^{L(R)}(\tau) \qquad \left(0 \le \mathfrak{r}^{L(R)}(t) \le 1\right)$$

**Probability** that the jump from site m to left(right) side occurs during time $0 \sim t$ in absence of jump to right(left) side.

## Inverse Transform Sampling



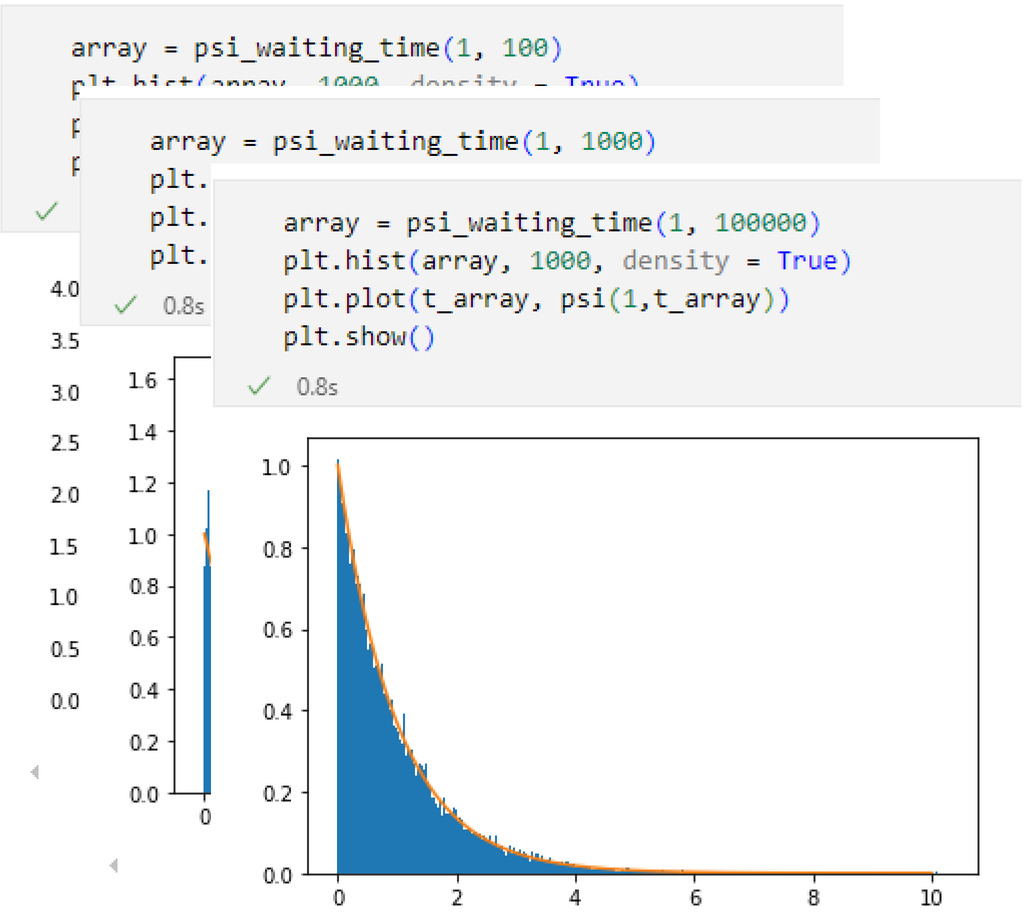

1. [0,1] 구간에서 균등 분포를 따르는 난수 U 를 생성한다.
2. 이 U를 이용해, $U = \mathfrak{r}_m^{L(R)}(t)$를 만족하는 $t$를 구한다.
3. 이 $t$는 $\psi_m^{L(R)}(t)$을 따른다.

**Proof? -> Do it yourself!**

[Google colab code link]

# Supplementary Material: How to make simulation code?

```python
import numpy as np
import matplotlib.pyplot as plt

def psi(k,t):
    return k*np.exp(-k*t)

t_array = np.linspace(0,10,1000)

def psi_waiting_time(k, nr):
    """
    Generates a random time from the waiting time distribution psi(t) = k e^-kt.

    Repeat nr times.

    Args:
        k: The rate parameter of the distribution.
        nr: The number of times to repeat the process.

    Returns:
        A random time array generated from the distribution.
    """
    return -np.log(np.random.uniform(0, 1,nr)) / k
```




$$U = \int_0^t d\tau \, k e^{-k\tau} = 1 - e^{-kt} \rightarrow t = -\frac{\ln(1-U)}{k}$$

[Google colab code link]

# Supplementary Material: How to make simulation code?

```python
def psi_win(kL,kR,nr):
    left = psi_waiting_time(kL, nr)
    right = psi_waiting_time(kR, nr)

    return right * (left > right) + left * (left < right)
```

$$\Psi(t) = (k_L + k_R)e^{-(k_L+k_R)t}$$

`(left > right) or (left < right)`: **Boolean Array**

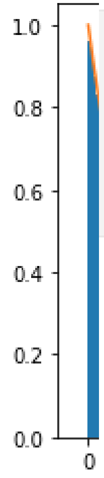
e.g. `left = [1,2,3,4], right = [4,3,2,1]`

→ `(left > right) = [0,0,1,1], (left < right) = [1,1,0,0]`

→ Return value = `[4,3,3,4]`

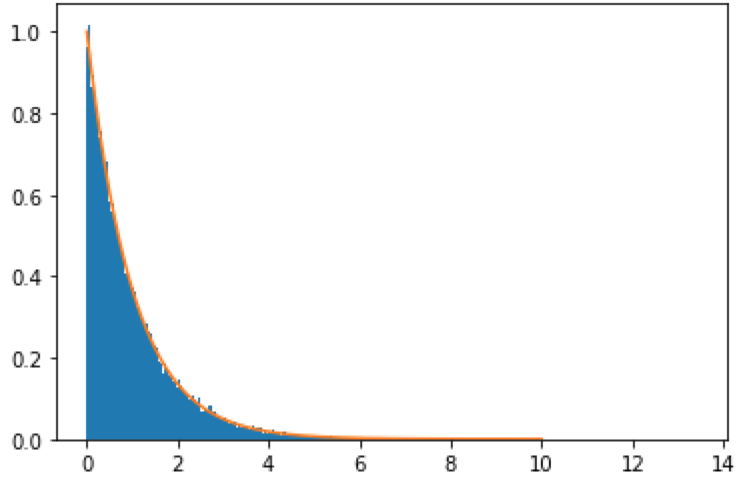
```python
array = psi_win(0.5, 0.5, 100000)
plt.hist(array, 1000, density = True)
plt.plot(t_array, psi(1,t_array))
plt.show()
```

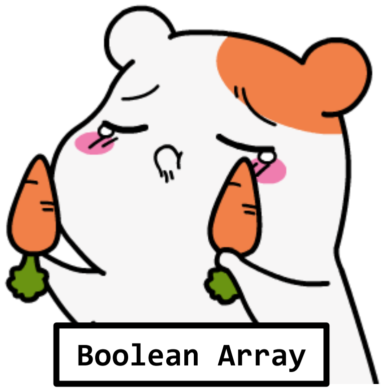
✓ 0.8s




```python
array = psi_win(0.2, 0.8, 100000)
plt.hist(array, 1000, density = True)
plt.plot(t_array, psi(1,t_array))
plt.show()
```

✓ 0.9s




[Google colab code link]

# Supplementary Material: How to make simulation code?

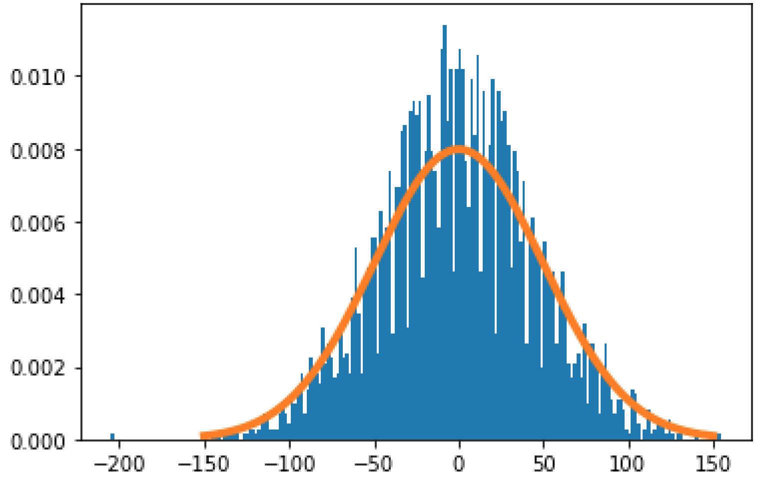
```python
def random_walker(kL,kR,m,t):
    """
    Args:
    kL, kR: rate coefficient
    m: initial position
    t: measuring time (final)

    Returns:
        position of random walker which starts at lattice site m.

    """
     time = 0
     position = m

     while time < t:
         tL = psi_waiting_time(kL, 1)[0]
         tR = psi_waiting_time(kR, 1)[0]

         if tL < tR:
             position -= 1
             time += tL
         elif tR < tL:
             position += 1
             time += tR

     return position
```

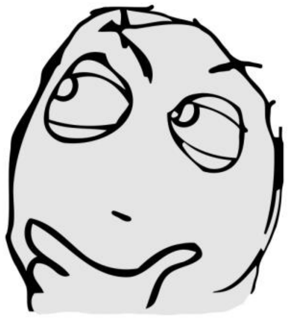
```python
nr = 4000
time = 1000

results = np.zeros(nr)

for i in range(0,nr):
    results[i] = random_walker(1,1,0,time)
```

✓ 59.6s

```python
plt.hist(results, 200, density = True)
plt.plot(x_axis, normal_distribution(50,x_axis,0), lw = 4)
plt.plot()
plt.show()
```

✓ 0.2s



# Supplementary Material: How to make simulation code?

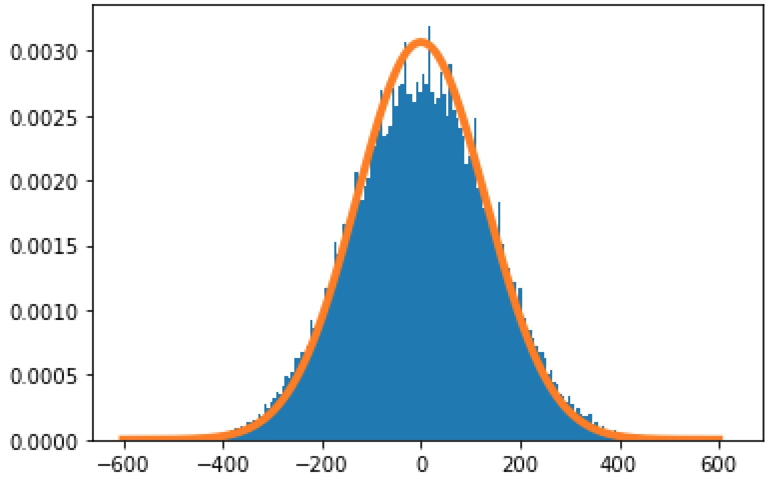
```
nr = 4000
time = 1000
✓ 59.6s
```

```python
def random_walker_parallel(kL,kR,m,t,nr):
    """
    Args:
    kL, kR: rate coefficient
    m: initial position
    t: measuring time (final)
    nr: number of repeatation.

    Returns:
        positions of random walkers which starts at lattice site m.

    """
    time = np.zeros(nr)
    position = m*np.ones(nr)

    while True:
        tL = psi_waiting_time(kL, nr)
        tR = psi_waiting_time(kR, nr)

        B_left = (tL < tR)
        B_right = (tL > tR)

        time += tL * B_left + tR * B_right
        B_time = (time < t)


        position -= B_left * B_time
        position += B_right * B_time

        if np.sum(B_time) == 0:
            break

    return position
```

```python
nr = 100000
time = 10000

results = random_walker_parallel(1,1,0,time, nr)
```
✓ 1m 6.3s

**Matrix calculation**은 code를 빠르게 만들어줘요.
**(python, matlab)**

```python
x_axis = np.linspace(-600,600,1201)

plt.hist(results, 200, density = True)
plt.plot(x_axis, normal_distribution(130,x_axis,0), lw = 4)
plt.plot()
plt.show()
```
✓ 0.1s




[Google colab code link]

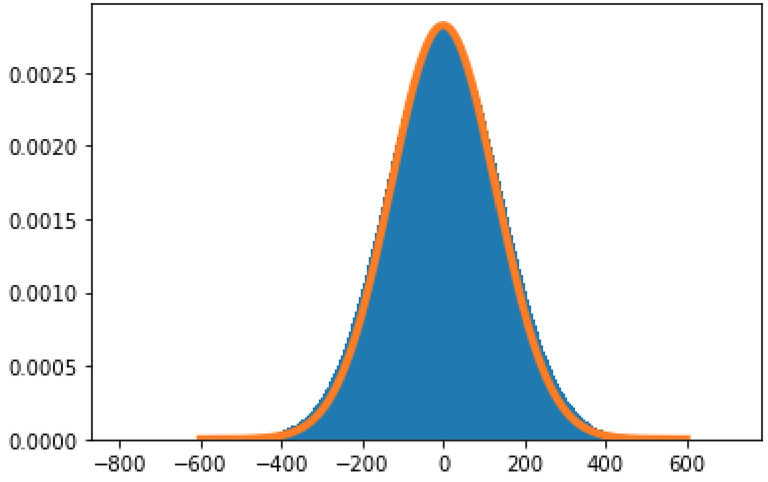
# Supplementary Material: How to make simulation code?

```python
def random_walker_parallel(kL,kR,m,t,nr):
    """
    Args:
    kL, kR: rate coefficient
    m: initial position
    t: measuring time (final)
    nr: number of repeatation.

    Returns:
        positions of random walkers which starts at lattice site m.

    """
    time = np.zeros(nr)
    position = m*np.ones(nr)

    while True:
        tL = psi_waiting_time(kL, nr)
        tR = psi_waiting_time(kR, nr)

        B_left = (tL < tR)
        B_right = (tL > tR)

        time += tL * B_left + tR * B_right
        B_time = (time < t)


        position -= B_left * B_time
        position += B_right * B_time

        if np.sum(B_time) == 0:
            break

    return position
```

**더 많이, 더 긴 시간까지 보면 Gaussian distribution에 수렴해요.**
**이는 Central Limit Theorem (CLT)로 증명 가능해요.**

```python
nr = 10000000
time = 10000

results_many = random_walker_parallel(1,1,0,time, nr)
```
✓ 174m 50.2s

```python
x_axis = np.linspace(-600,600,1201)

plt.hist(results_many, 301, density = True)
plt.plot(x_axis, 0.92*normal_distribution(130,x_axis,0), lw = 4)
plt.plot()
plt.show()
```
✓ 0.3s



# Supplementary Material: Our friend, ChatGPT & Claude

"인간이 보유한 지식 전부를 포함하는 하나의 원을 상상해봅시다"

Imagir

"대학을 졸업하고 학사 학위 학위를 취득하면 당신은 전공 분야를 갖게 됩니다."

With a bachelor's degree, you gain a specialty:

"석사 학위를 따면서 전공을 더 깊이 파고들게 되죠."

A master's degree deepens that specialty:

"(박사를 따기 위해) 논문을 읽어나가다 보면 당신은 인간 지식의 최전선에 다다릅니다."

Reading research papers takes you to the edge of human knowledge:

"한 분야의 경계선에 다다르면, 당신은 (특정 분야에) 더욱 초점을 맞추게 됩니다."

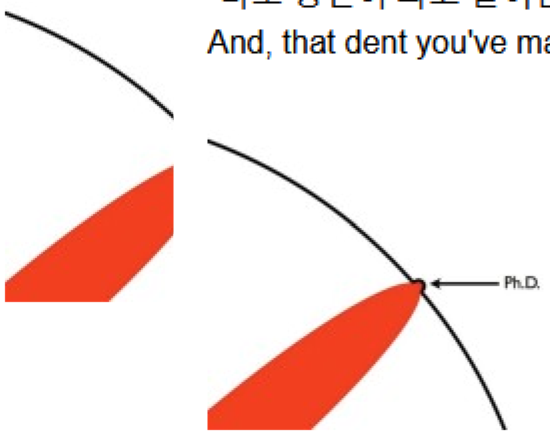
Once you're at the boundary, you focus:

"마침내 어느 날, 그 경계에 새 길이 생겼음을 알게 됩니다."

Until one day, the boundary gives way:

"바로 당신이 파고 들어간 움푹 들어간 곳을 사람들은 박사학위라고 부르죠."

And, that dent you've made is called a Ph.D.:




ChatGPT

Claude

BY ANTHROP\C

# Supplementary Material: Our friend, ChatGPT & Claude

**TA. Han's homework**

Consider the motion of Harmonic Oscillator, the position and velocity can be expressed by a function in terms of time $t$, $X(t)$ and $V(t)$. And the restoring force that the body feels is defined as following.

$F = m\frac{d^2X(t)}{dt^2} = -kX(t)$ When $t=0$, $X(0)=x_0$, $V(0)=v_0$.

3) Using 'Laplace Transform' method, obtain the $X(t)$.

Hint1 : Let, $\omega^2 = \frac{k}{m}$

Hint2 : $\mathcal{L}[\cos(\omega t)] = \frac{s}{s^2+\omega^2}$ , $\mathcal{L}[\sin(\omega t)] = \frac{\omega}{s^2+\omega^2}$

이 문제 한번 풀어줘

ME

To solve the given problem using the Laplace Transform method, follow these steps:

1. **Write the differential equation for the harmonic oscillator:**

$m\frac{d^2X(t)}{dt^2} + kX(t) = 0$

Given $\omega^2 = \frac{k}{m}$, we can rewrite this as:

$\frac{d^2X(t)}{dt^2} + \omega^2X(t) = 0$

2. **Apply the Laplace Transform to both sides of the equation:**

Using the properties of Laplace Transform, we have:

$\mathcal{L}\left\{\frac{d^2X(t)}{dt^2}\right\} + \omega^2\mathcal{L}\{X(t)\} = 0$

Recall that the Laplace Transform of a second derivative is given by:

$\mathcal{L}\left\{\frac{d^2X(t)}{dt^2}\right\} = s^2\mathcal{L}\{X(t)\} - sX(0) - \frac{dX(0)}{dt}$

3. **Substitute the initial conditions $X(0) = x_0$ and $\frac{dX(0)}{dt} = v_0$:**

$s^2\mathcal{L}\{X(t)\} - sx_0 - v_0 + \omega^2\mathcal{L}\{X(t)\} = 0$

4. **Solve for $\mathcal{L}\{X(t)\}$:**

$\mathcal{L}\{X(t)\}(s^2+\omega^2) = sx_0 + v_0$

$\mathcal{L}\{X(t)\} = \frac{sx_0+v_0}{s^2+\omega^2}$

5. **Find the inverse Laplace Transform:**

We use the given hints for the inverse Laplace Transform:

$\mathcal{L}^{-1}\left\{\frac{s}{s^2+\omega^2}\right\} = \cos(\omega t)$
$\mathcal{L}^{-1}\left\{\frac{\omega}{s^2+\omega^2}\right\} = \sin(\omega t)$

Using these, we get:

$X(t) = \mathcal{L}^{-1}\left\{\frac{sx_0+v_0}{s^2+\omega^2}\right\}$
$X(t) = x_0\cos(\omega t) + \frac{v_0}{\omega}\sin(\omega t)$

Therefore, the solution to the harmonic oscillator using the Laplace Transform method is:

$X(t) = x_0\cos(\omega t) + \frac{v_0}{\omega}\sin(\omega t)$

# Supplementary Material: Our friend, ChatGPT & Claude

**Homework 1:**

코드를 활용해서 $\psi(t) = ke^{-kt}$를 따르는 waiting time들을 생성하고,

이 시간의 평균($\langle t \rangle$)과 분산($\langle t^2 \rangle - \langle t \rangle^2$)을 구해 보시오.

생성된 random time의 개수가 커짐에 따라 이 평균과 분산이 어떤 값으로 가는지 확인해보세요.

**Homework 2:**

코드를 활용해서 $\psi(t) = ke^{-kt}$를 따르는 waiting time를 n개 생성한 뒤, 이 n개의 시간을 합해보시오.

이렇게 생성된 시간 $\mathrm{T} = \sum_{i=1}^{n} t_i$의 distribution을 그려보고, 생각한 distribution을 따르는지 확인해보시오.

Hint: $\left(\hat{\psi}(u)\right)^n = \left(\frac{k}{u+k}\right)^n$

# Supplementary Material: Our friend, ChatGPT & Claude

**Homework 3:**

$$E + S \underset{k_{-1}}{\overset{k_1}{\rightleftharpoons}} ES \xrightarrow{k_2} E + P$$

앞에서 사용한 simulation 방법을 활용한다 했을 때, 초기($t = 0$)에 $S$로 존재하던 물질이 $P$가 되는데 걸리는 시간을 어떻게 추출할 수 있을 지 한번 생각해보시오.

이 시간들을 얻어 낸 뒤 이를 이용해 distribution을 그렸을 때, 생각한 distribution을 따르는지 확인해보시오.

(Homework 1, 2는 필수, 3은 advance입니다. 그리고 사실 homework 1, 2는 제가 드리는 파일에 답이 이미...)

Google Colaboratory

ChatGPT

Claude

BY ANTHROP\C